11.2 Balanced Search Trees

In the closing of the previous section, we noted that if we could assume a random series of insertions and removals, the standard binary search tree supports $O(\log n)$ expected running times for the basic map operations. However, we may only claim $O(n)$ worst-case time, because some sequences of operations may lead to an unbalanced tree with height proportional to n.

In the remainder of this chapter, we explore four search tree algorithms that provide stronger performance guarantees. Three of the four data structures (AVL trees, splay trees, and red-black trees) are based on augmenting a standard binary search tree with occasional operations to reshape the tree and reduce its height.

The primary operation to rebalance a binary search tree is known as a ***rotation***. During a rotation, we "rotate" a child to be above its parent, as diagrammed in Figure 11.8.

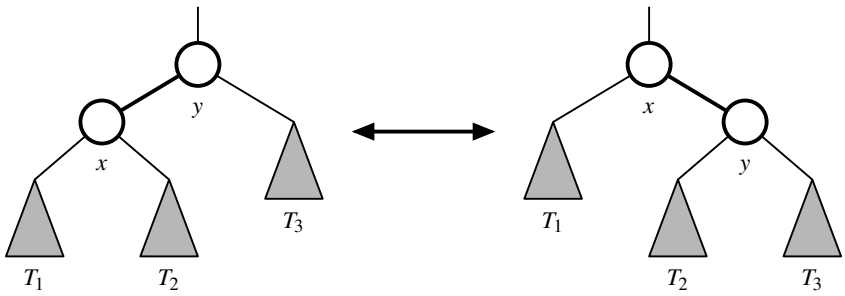


Figure 11.8: A rotation operation in a binary search tree. A rotation can be performed to transform the left formation into the right, or the right formation into the left. Note that all keys in subtree T_1 have keys less than that of position x, all keys in subtree T_2 have keys that are between those of positions x and y, and all keys in subtree T_3 have keys that are greater than that of position y.

To maintain the binary search tree property through a rotation, we note that if position x was a left child of position y prior to a rotation (and therefore the key of x is less than the key of y), then y becomes the *right* child of x after the rotation, and vice versa. Furthermore, we must relink the subtree of items with keys that lie between the keys of the two positions that are being rotated. For example, in Figure 11.8 the subtree labeled T_2 represents items with keys that are known to be greater than that of position x and less than that of position y. In the first configuration of that figure, T_2 is the right subtree of position x; in the second configuration, it is the left subtree of position y.

Because a single rotation modifies a constant number of parent-child relationships, it can be implemented in $O(1)$ time with a linked binary tree representation.

In the context of a tree-balancing algorithm, a rotation allows the shape of a tree to be modified while maintaining the search tree property. If used wisely, this operation can be performed to avoid highly unbalanced tree configurations. For example, a rightward rotation from the first formation of Figure 11.8 to the second reduces the depth of each node in subtree T_1 by one, while increasing the depth of each node in subtree T_3 by one. (Note that the depth of nodes in subtree T_2 are unaffected by the rotation.)

One or more rotations can be combined to provide broader rebalancing within a tree. One such compound operation we consider is a ***trinode restructuring***. For this manipulation, we consider a position x, its parent y, and its grandparent z. The goal is to restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees. Pseudo-code for a restructure(x) method is given in Code Fragment 11.9 and illustrated in Figure 11.9. In describing a trinode restructuring, we temporarily rename the positions x, y, and z as a, b, and c, so that a precedes b and b precedes c in an inorder traversal of T. There are four possible orientations mapping x, y, and z to a, b, and c, as shown in Figure 11.9, which are unified into one case by our relabeling. The trinode restructuring replaces z with the node identified as b, makes the children of this node be a and c, and makes the children of a and c be the four previous children of x, y, and z (other than x and y), while maintaining the inorder relationships of all the nodes in T.

Algorithm restructure(x):

Input: A position x of a binary search tree T that has both a parent y and a grandparent z

Output: Tree T after a trinode restructuring (which corresponds to a single or double rotation) involving positions x, y, and z

1: Let (a, b, c) be a left-to-right (inorder) listing of the positions x, y, and z, and let (T_1, T_2, T_3, T_4) be a left-to-right (inorder) listing of the four subtrees of x, y, and z not rooted at x, y, or z.
2: Replace the subtree rooted at z with a new subtree rooted at b.
3: Let a be the left child of b and let T_1 and T_2 be the left and right subtrees of a, respectively.
4: Let c be the right child of b and let T_3 and T_4 be the left and right subtrees of c, respectively.

Code Fragment 11.9: The trinode restructuring operation in a binary search tree.

In practice, the modification of a tree T caused by a trinode restructuring operation can be implemented through case analysis either as a single rotation (as in Figure 11.9a and b) or as a double rotation (as in Figure 11.9c and d). The double rotation arises when position x has the middle of the three relevant keys and is first rotated above its parent, and then above what was originally its grandparent. In any of the cases, the trinode restructuring is completed with $O(1)$ running time.

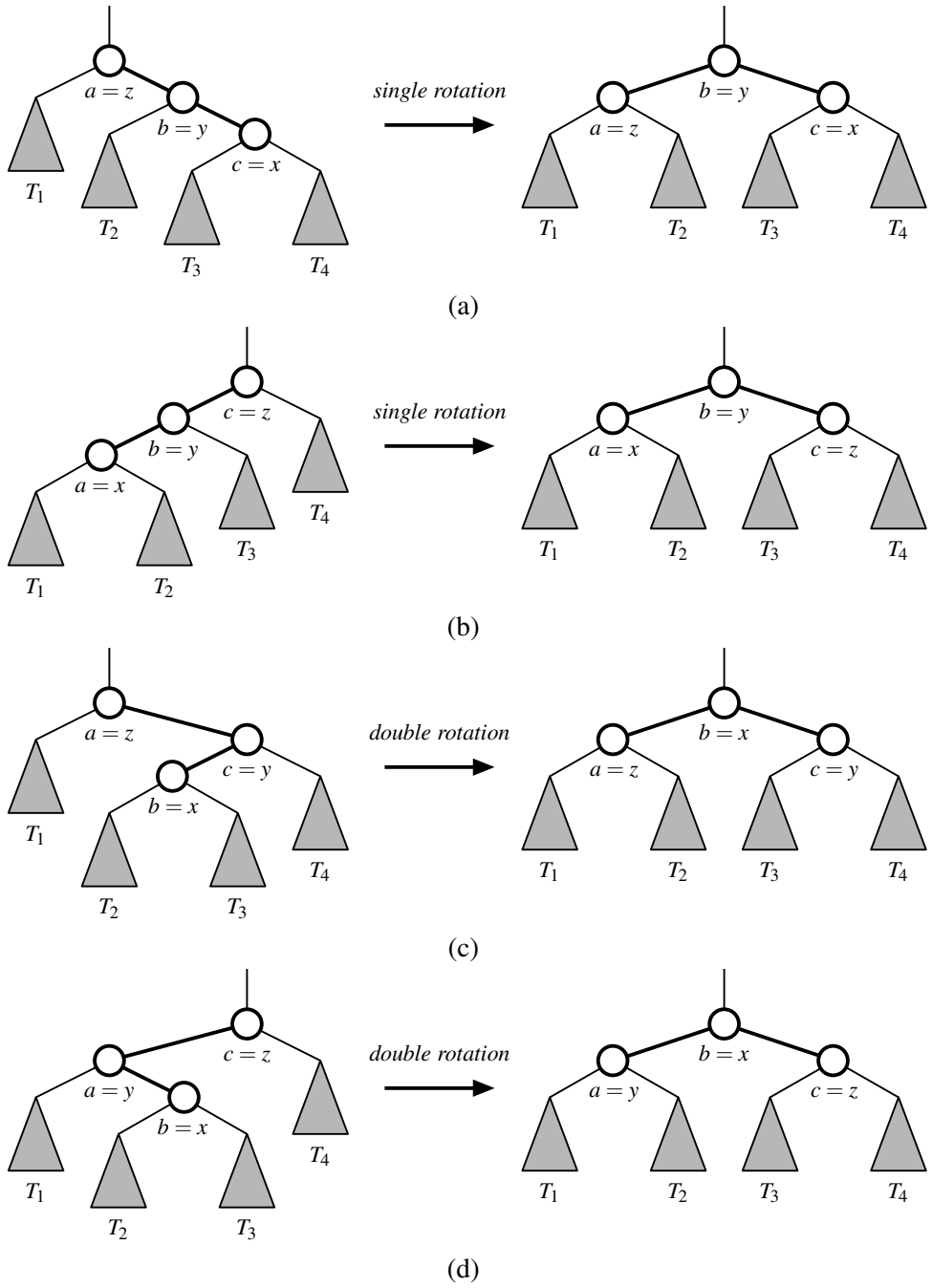


Figure 11.9: Schematic illustration of a trinode restructuring operation: (a and b) require a single rotation; (c and d) require a double rotation.

11.2.1 Python Framework for Balancing Search Trees

Our TreeMap class, introduced in Section 11.1.4, is a concrete map implementation that does not perform any explicit balancing operations. However, we designed that class to also serve as a base class for other subclasses that implement more advanced tree-balancing algorithms. A summary of our inheritance hierarchy is shown in Figure 11.10.

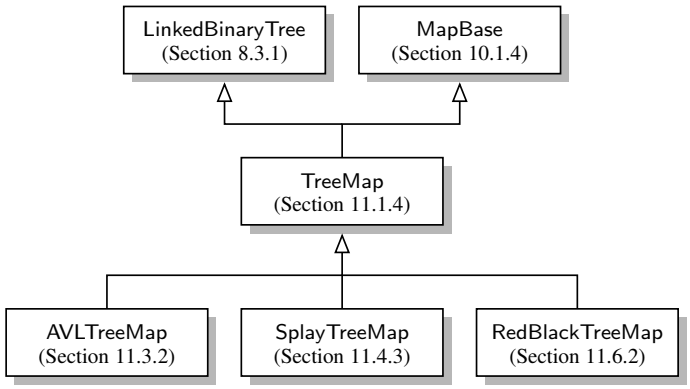


Figure 11.10: Our hierarchy of balanced search trees (with references to where they are defined). Recall that TreeMap inherits multiply from LinkedBinaryTree and MapBase.

Hooks for Rebalancing Operations

Our implementation of the basic map operations in Section 11.1.4 includes strategic calls to three nonpublic methods that serve as ***hooks*** for rebalancing algorithms:

- A call to _rebalance_insert(p) is made from within the __setitem__ method immediately after a new node is added to the tree at position *p*.
- A call to _rebalance_delete(p) is made each time a node has been deleted from the tree, with position *p* identifying the *parent* of the node that has just been removed. Formally, this hook is called from within the public delete(p) method, which is indirectly invoked by the public __delitem__(k) behavior.
- We also provide a hook, _rebalance_access(p), that is called when an item at position *p* of a tree is accessed through a public method such as __getitem__. This hook is used by the ***splay tree*** structure (see Section 11.4) to restructure a tree so that more frequently accessed items are brought closer to the root.

We provide trivial declarations of these three methods, in Code Fragment 11.10, having bodies that do nothing (using the **pass** statement). A subclass of TreeMap may override any of these methods to implement a nontrivial action to rebalance a tree. This is another example of the ***template method design pattern***, as seen in Section 8.4.6.

```
174    def _rebalance_insert(self, p): pass
175    def _rebalance_delete(self, p): pass
176    def _rebalance_access(self, p): pass
```

Code Fragment 11.10: Additional code for the TreeMap class (continued from Code Fragment 11.8), providing stubs for the rebalancing hooks.

Nonpublic Methods for Rotating and Restructuring

A second form of support for balanced search trees is our inclusion of nonpublic utility methods _rotate and _restructure that, respectively, implement a single rotation and a trinode restructuring (described at the beginning of Section 11.2). Although these methods are not invoked by the public TreeMap operations, we promote code reuse by providing these implementation in this class so that they are inherited by all balanced-tree subclasses.

Our implementations are provided in Code Fragment 11.11. To simplify the code, we define an additional _relink utility that properly links parent and child nodes to each other, including the special case in which a "child" is a None reference. The focus of the _rotate method then becomes redefining the relationship between the parent and child, relinking a rotated node directly to its original grandparent, and shifting the "middle" subtree (that labeled as T_2 in Figure 11.8) between the rotated nodes. For the trinode restructuring, we determine whether to perform a single or double rotation, as originally described in Figure 11.9.

Factory for Creating Tree Nodes

We draw attention to an important subtlety in the design of both our TreeMap class and the original LinkedBinaryTree subclass. The low-level definition of a node is provided by the nested _Node class within LinkedBinaryTree. Yet, several of our tree-balancing strategies require that auxiliary information be stored at each node to guide the balancing process. Those classes will override the nested _Node class to provide storage for an additional field.

Whenever we add a new node to the tree, as within the _add_right method of the LinkedBinaryTree (originally given in Code Fragment 8.10), we intentionally instantiate the node using the syntax self._Node, rather than the qualified name LinkedBinaryTree._Node. This is vital to our framework! When the expression self._Node is applied to an instance of a tree (sub)class, Python's name resolution follows the inheritance structure (as described in Section 2.5.2). If a subclass has overridden the definition for the _Node class, instantiation of self._Node relies on the newly defined node class. This technique is an example of the ***factory method design pattern***, as we provide a *subclass* the means to control the type of node that is created within methods of the *parent* class.

```
177   def _relink(self, parent, child, make_left_child):
178     """Relink parent node with child node (we allow child to be None)."""
179     if make_left_child:                          # make it a left child
180       parent._left = child
181     else:                                        # make it a right child
182       parent._right = child
183     if child is not None:                        # make child point to parent
184       child._parent = parent
185
186   def _rotate(self, p):
187     """Rotate Position p above its parent."""
188     x = p._node
189     y = x._parent                                # we assume this exists
190     z = y._parent                                # grandparent (possibly None)
191     if z is None:
192       self._root = x                             # x becomes root
193       x._parent = None
194     else:
195       self._relink(z, x, y == z._left)           # x becomes a direct child of z
196     # now rotate x and y, including transfer of middle subtree
197     if x == y._left:
198       self._relink(y, x._right, True)            # x._right becomes left child of y
199       self._relink(x, y, False)                  # y becomes right child of x
200     else:
201       self._relink(y, x._left, False)            # x._left becomes right child of y
202       self._relink(x, y, True)                   # y becomes left child of x
203
204   def _restructure(self, x):
205     """Perform trinode restructure of Position x with parent/grandparent."""
206     y = self.parent(x)
207     z = self.parent(y)
208     if (x == self.right(y)) == (y == self.right(z)):  # matching alignments
209       self._rotate(y)                            # single rotation (of y)
210       return y                                   # y is new subtree root
211     else:                                        # opposite alignments
212       self._rotate(x)                            # double rotation (of x)
213       self._rotate(x)
214       return x                                   # x is new subtree root
```

Code Fragment 11.11: Additional code for the TreeMap class (continued from Code Fragment 11.10), to provide nonpublic utilities for balanced search tree subclasses.